

LAB 5

מגישות :

לירון עדי 322528787

שחר גולומבק 315099614

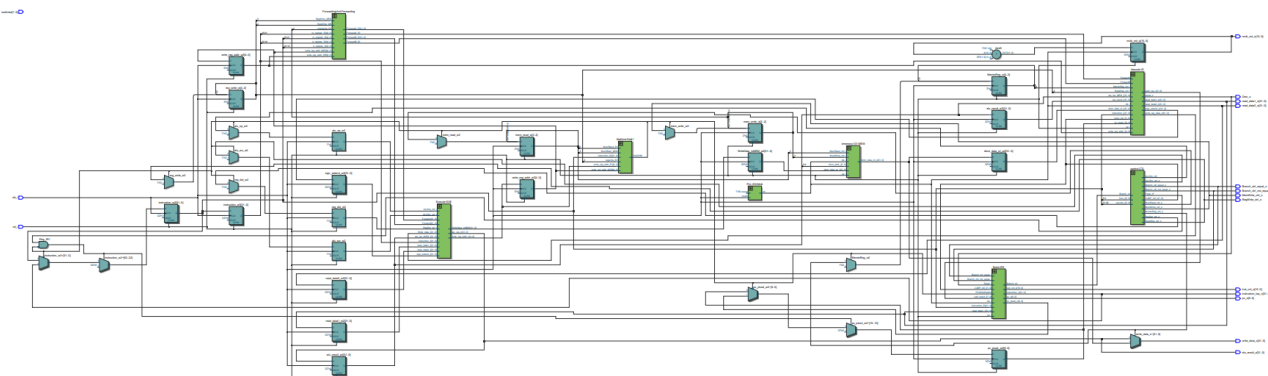
מערכת ה – pipeline

כעת נתאר את המערכת במבט כולל, ונפרט את תתי-המודולים המרכיבים אותה. עבור כל מודול נציג בקצרה את עקרונות הפעולה שלו, נצרף את תרשים ה-RTL לאחר סינתזה, נסביר את הלוגיקה שבה נעשה שימוש, ונציין את הנתיב הקריטי הרלוונטי. בנוסף לכך, נדגים כיצד המודול משתלב בפרויקט בסביבת Quartus.

מצורף כמיוות של כל השימושים של המערכת הכוללת.

Analysis & Synthesis Resource Utilization by Entity							
<<Filter>>							
	Compilation Hierarchy Node	Combinational ALUTs	Dedicated Logic Registers	Block Memory Bits	DSP Blocks	Pins	Virtual Pins
1	▼ MIPS	2045 (57)	1566 (301)	16384	1	217	0
1	▸ Execute:EXE	842 (586)	0 (0)	0	1	0	0
2	▸ Forwarding...Forwarding	9 (9)	0 (0)	0	0	0	0
3	▸ Idecode:ID	851 (851)	992 (992)	0	0	0	0
4	▸ Ifetch:IFE	105 (58)	97 (33)	8192	0	0	0
5	▸ PLL:\G0:Mclk	0 (0)	0 (0)	0	0	0	0
6	▸ StallUnit:Stall1	14 (14)	0 (0)	0	0	0	0
7	▸ control:CTL	10 (10)	0 (0)	0	0	0	0
8	▸ dmemory:\G1:MEM	47 (0)	64 (0)	8192	0	0	0
9	▸ sld_hub:auto_hub	110 (1)	112 (0)	0	0	0	0

מצורף הRTL של המערכת הכוללת (שכבת MIPS) לאחר סינתזה בQuartus :



בנוסף צירפנו סיגנלי כניסה ומוצא לצורך דיבוג ובקרה על התוצאות:

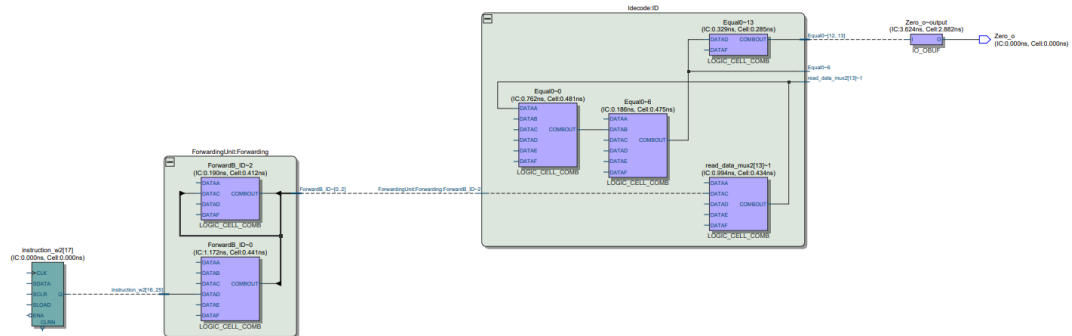
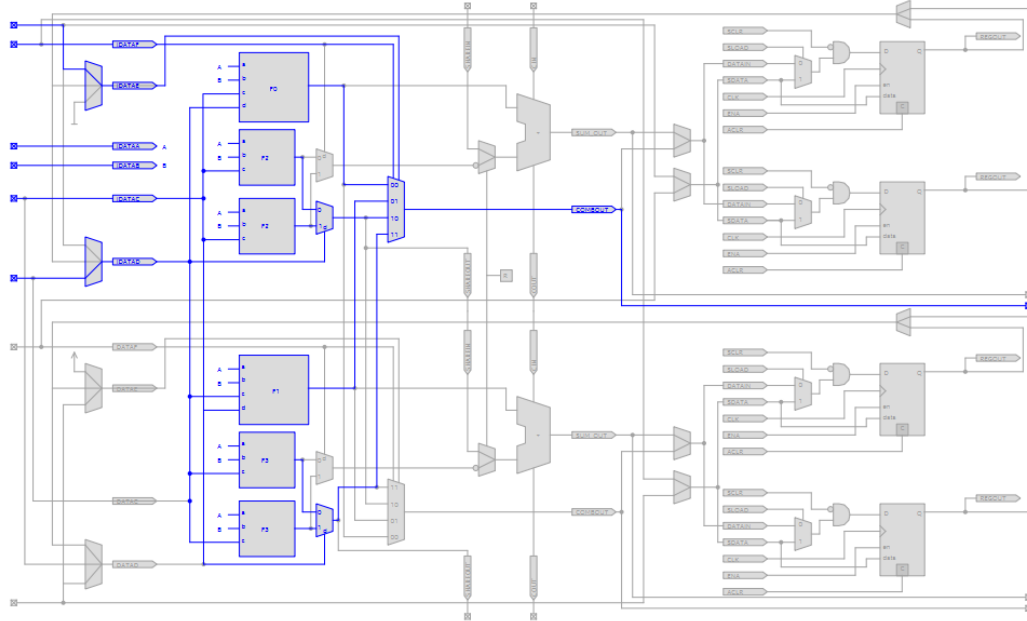
BPADDR_i – לצורך טריגר של המערכת על פקודה רצויה.

STCNT_o, FHCNT_o – מוצאים אשר סופרים את כמות הstall וה flush על מנת לחשב את הIPC (הביצועים) של המערכת עבור קודים שהרצנו במערכת זו.

נתבון בתדר המקסימלי שקיבלנו עבור לוגיקת Pipline:

	Fmax	Restricted Fmax	Clock Name	Note
1	43.46 MHz	43.46 MHz	\G0:Mclk altpll_component auto_generated generic_pll1~PLL_OUTPUT_COUNTER divclk	
2	86.63 MHz	86.63 MHz	clk_i	

נתבון גם בנתיב הקריטי:



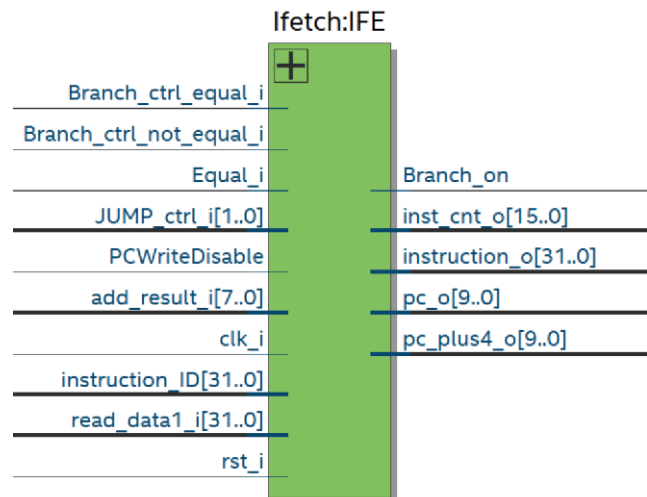
כעת נעבור לסקירה מפורטת של כל אחד מהשלבים:

שלב הIFETCH

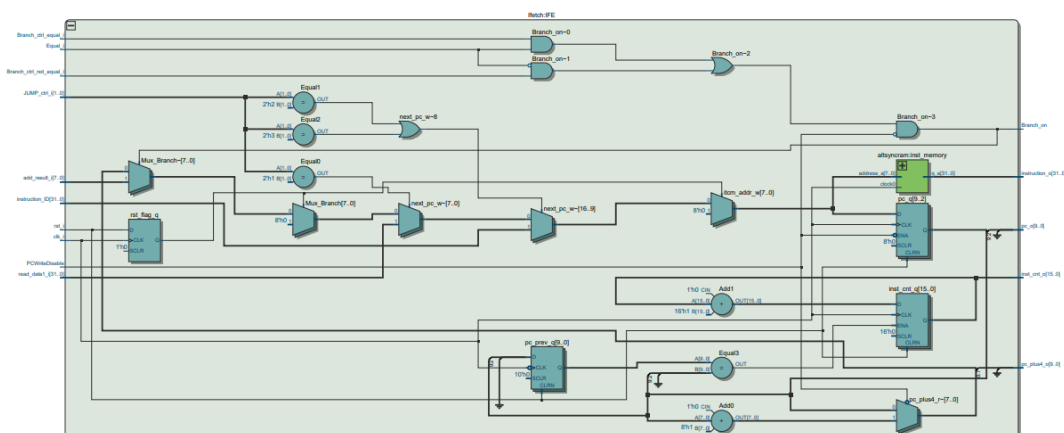
פעולת המודול – באמצעות כתובת הPC הבאת פקודה מה – instruction memory . והעברה לשלב decoden. בנוסף בשלב זה נברור מה תייה הכתובת הבאה של הPC על סמך הפקודות הקודמות (למשל: branch, jump)

במקביל לשלב ה IFETCH קיים רכיב ה stall unit שאחראי על השהיית המערכת במקרים בהם אין אפשרות לקדם את המערכת מבלי להרוס את התוצאה הנדרשת.

שרטוט ENTITY -



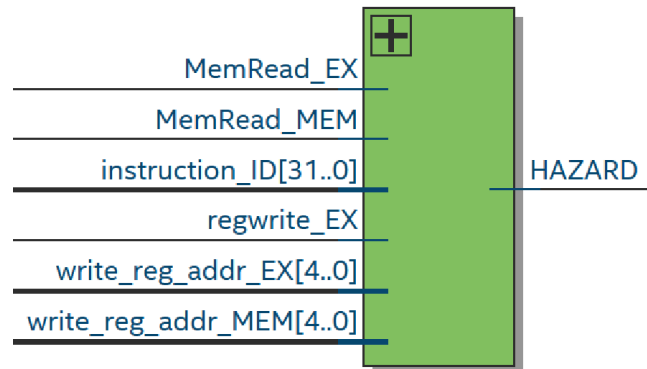
שרטוט RTL של המודול –



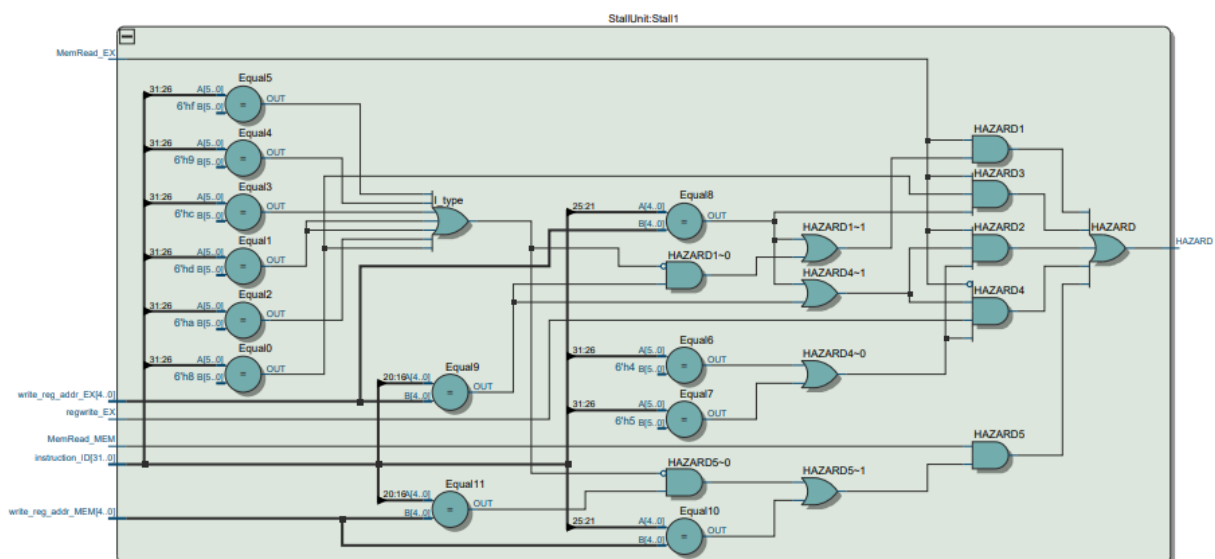
STALL UNIT

שרטוט ENTITY

StallUnit:Stall1



שרטוט RTL של המודול



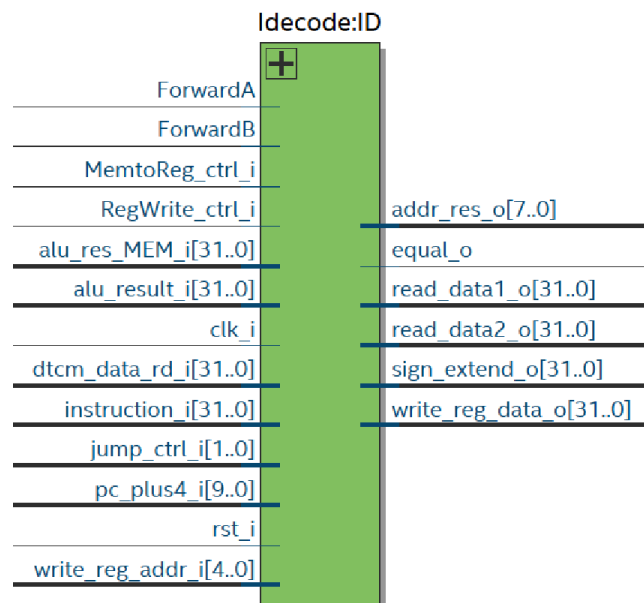
שלב WB + I-DECODE

פעולת המודול – בשלב זה נלקחת הפקודה משלב ה fetch עוברת פיענוח לאחד משלוש סוגי הפעולות I-TYPE, R-TYPE, J-TYPE, וחלקי הפקודה מופרדים ונכנסים לתוך ה Register file שם ניתן לקרוא את המידע הרלוונטי לפקודה במידת הצורך, או לכתוב לתוכו בשלב ה – WB.

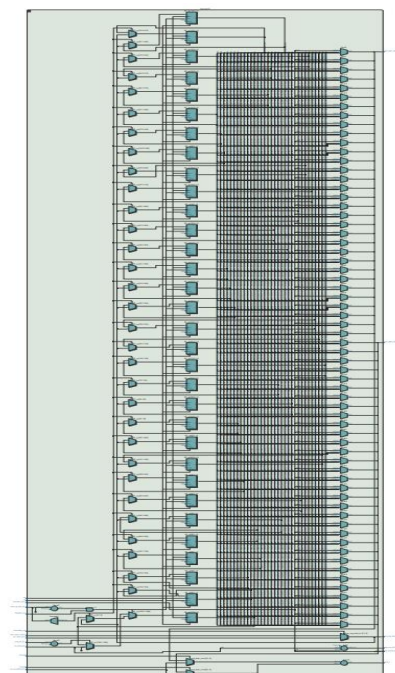
בתוך מודול זה מתקיים שני שלבים שונים במקביל שבעזרת טיפול נכון לא מתנגשים.

בשלב זה מתבצעת בדיקה עבור פעולת ה branch שתחזיר תוצאה ישירות לשלב ה/IFETCH

שרטוט ENTITY -



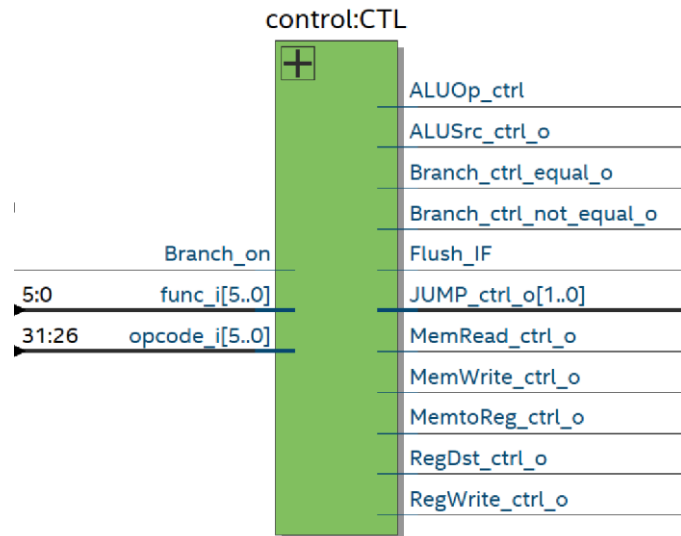
שרטוט RTL של המודול –



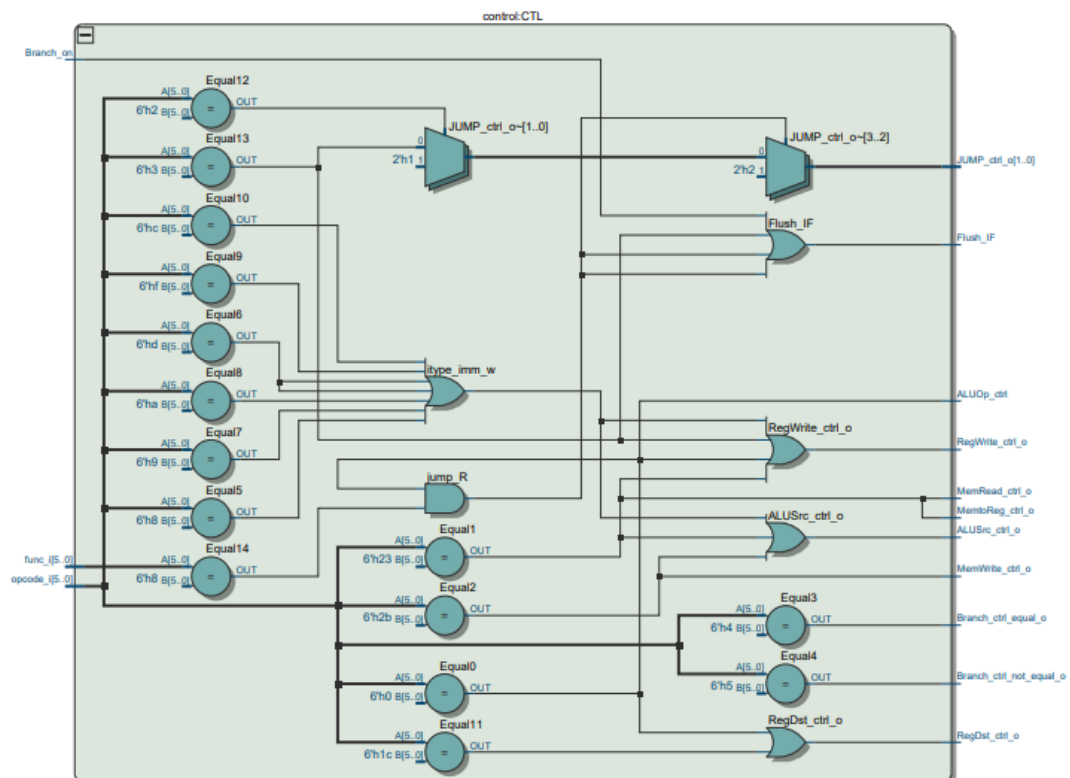
CONTROL UNIT – ה

פעולת המודול – רכיב זה אחרי על קווי הבקרה של כל פקודה. נקבל משלב IFETCH את הפקודה ונסווג את קווי הבקרה הנדרשים לפי הצורך.

שרטוט ENTITY –



שרטוט RTL של המודול –

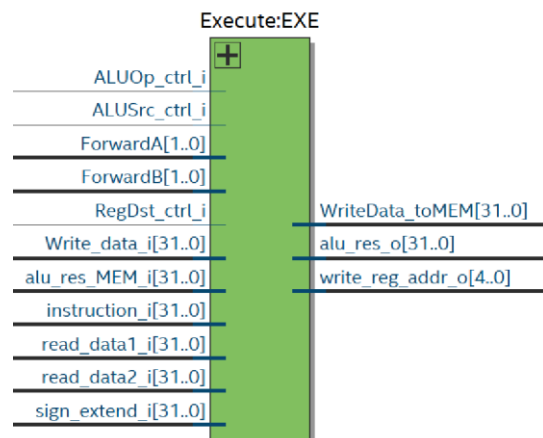


שלב EXECUTE

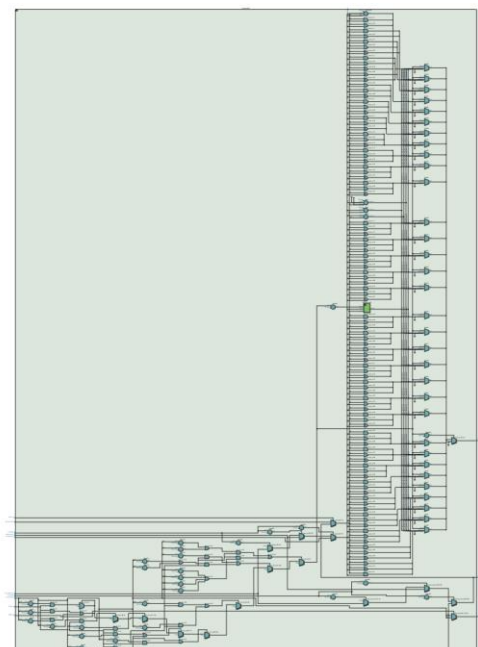
פעולת המודול – זהו השלב שמגיע אחרי DECODE. בשלב זה מתבצע חישוב ברכיב ה-ALU עבור פקודות Rtype ו-Itype. על מנת להגדיר את סוג הפעולה שאותה נרצה שה-ALU יבצע נשתמש בשלב זה ב-ALU control שמשמש בקו בקרה שיוצא מה control unit ובחלקים של Instruction.

בשלב זה יש לנו טיפול ב-*data hazard* באמצעות רכיב ה-*forwarding unit* שעבור מקרים שבהם נדרש שימוש בתוצאה של שלב מתקדם שעדיין לא הגיעה לאחור. בעצם נעביר את המידע לאחור לתוך שלב זה על מנת להשתמש בו עוד לפני שהגיע ליעד שלו.

שרטוט ENTITY -

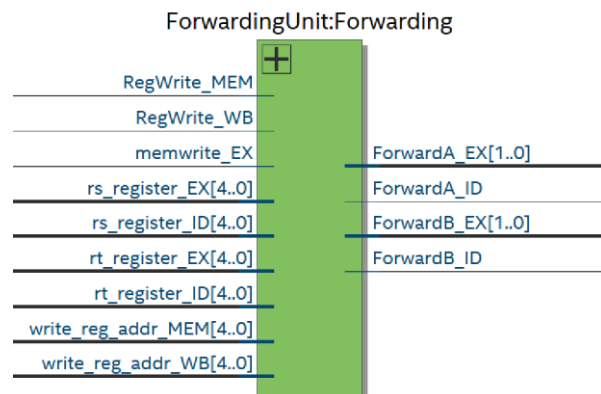


שרטוט RTL של המודול -

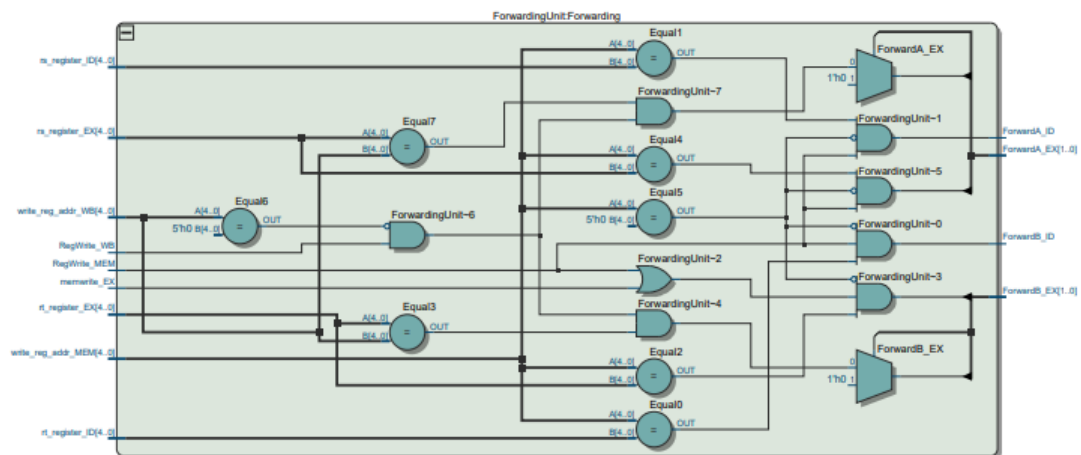


FORWARDING UNIT

– שרטוט ENTITY



שרטוט RTL של המודול -

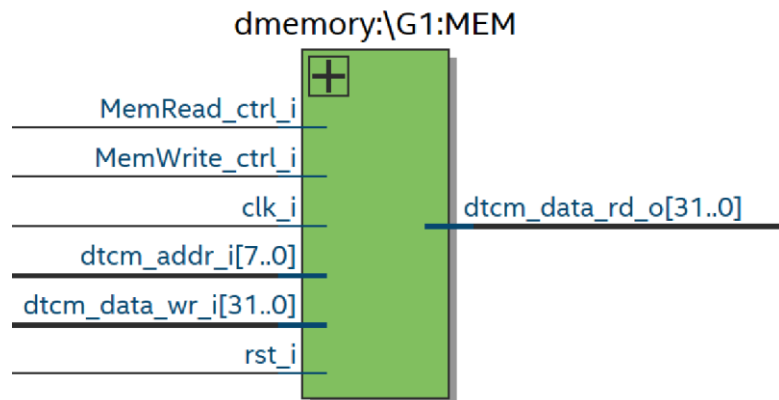


שלב ה - DATA MEMORY

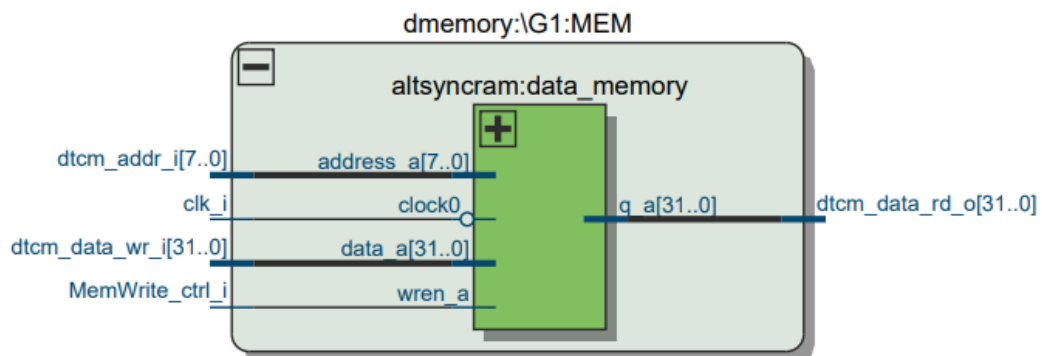
פעולת המודול – מודול זה בדומה לRF משמש לקריאה וכתיבה של מידע כאשר רכיב זה חיצוני למעבד ומכיל כמות גדולה בהרבה של מידע , ומסוגל להכיל מידע גדול יותר.

שלב זה נמצא ישר אחרי שלב הEXECUTE ומעביר את המידע במידת הצורך לשלב הWRITE BACK .

שרטוט ENTITY -



שרטוט RTL של המודול -



הרצנו את ה - signal tap על test2 מתוך הטסים שקיבלנו לדוגמא, להלן הקוד באסמבלי:

.data

arr1: .word 1,2,3,4,5,6,7,8,

arr2: .word 100,99,98,97,96,95,94,93,

res1: .space 32 # SIZE*4=32[Byte] - ADD result array

res2: .space 32 # SIZE*4=32[Byte] - SUB result array

res3: .space 32 # SIZE*4=32[Byte] - MUL result array

SIZE: .word 8

.text

main:

li \$sp,0x01FC # stack initial address is 200

lw \$s0,SIZE(\$0) # s0 = SIZE

la \$t1,arr1 # t1 points to arr1

la \$t2,arr2 # t2 points to arr2

la \$s1,res1 # s1 points to res1

la \$s2,res2 # s2 points to res2

la \$s3,res3 # s3 points to res3

loop:

addi \$sp,\$sp,-16

sw \$s0,12(\$sp) # push SIZE

sw \$s1,8(\$sp) # push res1 pointer

sw \$t2,4(\$sp) # push arr2 pointer

sw \$t1,0(\$sp) # push arr1 pointer

jal mat_add

addi \$sp,\$sp,-16

sw \$s0,12(\$sp) # push SIZE

sw \$s2,8(\$sp) # push res2 pointer

sw \$t2,4(\$sp) # push arr2 pointer

sw \$t1,0(\$sp) # push arr1 pointer

jal mat_sub

```

        addi $sp,$sp,-16
        sw  $s0,12($sp)    # push SIZE
        sw  $s3,8($sp)      # push res3 pointer
        sw  $t2,4($sp)      # push arr2 pointer
        sw  $t1,0($sp)      # push arr1 pointer
        jal mat_mul

finish:    beq $zero,$zero,finish
#-----

mat_add:  lw  $t6,12($sp)    # push SIZE
          lw  $t5,8($sp)      # push res1 pointer
          lw  $t4,4($sp)      # push arr2 pointer
          lw  $t3,0($sp)      # push arr1 pointer

add_l:    lw  $t7,0($t3)
          lw  $t8,0($t4)
          lw  $t9,0($t5)

          add $t0,$t7,$t8
          sw  $t0,0($t5)      # Mem[s1] = res1[i]= arr1[i]+arr2[i]

          addi $t3,$t3,4
          addi $t4,$t4,4
          addi $t5,$t5,4
          addi $t6,$t6,-1
          bne $t6,$zero,add_l
          addi $sp,$sp,16
          jr  $ra
#-----

mat_sub:  lw  $t6,12($sp)    # push SIZE
          lw  $t5,8($sp)      # push res2 pointer
          lw  $t4,4($sp)      # push arr2 pointer
          lw  $t3,0($sp)      # push arr1 pointer

```

```

sub_l:      lw  $t7,0($t3)

            lw  $t8,0($t4)

            lw  $t9,0($t5)


            sub $t0,$t7,$t8

            sw  $t0,0($t5)          # Mem[s1] = res1[i]= arr1[i]-arr2[i]


            addi $t3,$t3,4
            addi $t4,$t4,4
            addi $t5,$t5,4
            addi $t6,$t6,-1
            bne $t6,$zero,sub_l
            addi $sp,$sp,16
            jr  $ra

```

#-----

```

mat_mul: lw  $t6,12($sp)          # push SIZE

            lw  $t5,8($sp)        # push res3 pointer
            lw  $t4,4($sp)        # push arr2 pointer
            lw  $t3,0($sp)        # push arr1 pointer

```

```

mul_l:      lw  $t7,0($t3)

            lw  $t8,0($t4)

            lw  $t9,0($t5)


            mul $t0,$t7,$t8

            sw  $t0,0($t5)          # Mem[s1] = res1[i]= arr1[i]*arr2[i]


            addi $t3,$t3,4
            addi $t4,$t4,4
            addi $t5,$t5,4
            addi $t6,$t6,-1
            bne $t6,$zero,mul_l
            addi $sp,$sp,16
            jr  $ra

```

#-----

ראינו שאכן מתעדכן הזיכרון:

000000	00 00 00 01	00 00 00 02	00 00 00 03	00 00 00 04	00 00 00 05
000005	00 00 00 06	00 00 00 07	00 00 00 08	00 00 00 09	00 00 00 0A
00000a	00 00 00 0B	00 00 00 0C	00 00 00 0D	00 00 00 0E	00 00 00 0F
00000f	00 00 00 10	00 00 00 11	00 00 00 12	00 00 00 13	00 00 00 14
000014	00 00 00 15	00 00 00 16	00 00 00 17	00 00 00 18	FF FF FF 9D
000019	FF FF FF 9F	FF FF FF A1	FF FF FF A3	FF FF FF A5	FF FF FF A7
00001e	FF FF FF A9	FF FF FF AB	00 00 00 64	00 00 00 C6	00 00 01 26
000023	00 00 01 84	00 00 01 E0	00 00 02 3A	00 00 02 92	00 00 02 E8
000028	00 00 00 08	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
00002d	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
000032	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00

ובהשוואה של הPC לפקודה האחרונה אנחנו אכן מגיעים לסוף התוכנית:

